SKIN CANCER CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK (CNN)

A Beginner-Friendly Machine Learning Project Report

Submitted by: [Your Full Name]
Registration / Roll Number: [Your Roll Number]
Program: BS Computer Science (BSCS)
Course: [Course Name]
Submitted to: [Instructor Name]
Department / University: [Department and University]
Submission Date: [Date]

Table of Contents

• Abstract

• 1. Introduction

• 2. Problem Statement

• 3. Objectives

• 4. Dataset Description

• 5. Exploratory Data Analysis (EDA)

• 6. Preprocessing and Dataset Split

• 7. CNN Architecture

• 8. Compilation and Training

• 9. Training Results

• 10. Evaluation Metrics

• 11. Sample Predictions

• 12. Limitations

• 13. Future Improvements

• 14. Conclusion

• References

Abstract

This project develops a Convolutional Neural Network (CNN) to classify dermoscopic skin-lesion images into nine dataset categories. The Kaggle notebook identified 2,357 images across nine classes. Images are resized to 128 × 128 pixels and normalized to the range 0–1. The collected image paths are split into training, validation, and testing subsets in a 70:10:20 ratio using stratified sampling. A CNN with convolutional, pooling, dense, and dropout layers is trained using Adam optimization. The displayed training output showed 60.28% training accuracy and 55.51% validation accuracy at that epoch. The final test accuracy must be copied from the notebook's evaluation output. This is an educational project, not a clinical diagnostic tool.

1. Introduction

Skin lesions can have different visual appearances, and image classification is a useful application of computer vision. Convolutional Neural Networks learn visual patterns such as edges, textures, shapes, and colour variations from images.

This project implements an introductory image-classification pipeline using Python and TensorFlow/Keras. It includes exploratory data analysis (EDA), preprocessing, CNN training, and model evaluation.

2. Problem Statement

The task is to predict the dataset label of a skin-lesion image from its visual content. The model receives an RGB image and produces probabilities for nine classes. The class with the highest probability is selected as the prediction.

The dataset is imbalanced, meaning some classes have more images than others. Therefore, per-class metrics should be reviewed in addition to overall accuracy.

3. Objectives

Explore the dataset and its class distribution.

Prepare images for deep-learning input.

Split images into 70% training, 10% validation, and 20% testing subsets.

Build and train a CNN for nine-class classification.

Plot training and validation accuracy and loss.

Evaluate the model using test accuracy, a classification report, a confusion matrix, and sample predictions.

4. Dataset Description

The dataset was accessed from Kaggle using the folder path configured in the notebook. The initial scan found 2,357 images and nine labels:

Actinic keratosis; basal cell carcinoma; dermatofibroma; melanoma; nevus; pigmented benign keratosis; seborrheic keratosis; squamous cell carcinoma; and vascular lesion.

The original dataset included folders such as Train and Test. To meet the assignment's requested 70/10/20 ratio, the notebook combined the discovered image paths into a dataframe and performed a stratified split across the collected images. State this clearly in the report because the final test subset was created from that dataframe rather than preserving the original folder split.

5. Exploratory Data Analysis (EDA)

EDA was used to count images, identify unique labels, inspect class counts, display sample images, and inspect image dimensions.

A class-distribution chart helps identify imbalanced categories. Displaying sample images checks that files can be opened and that labels correspond to their folders. Image-dimension inspection helps determine whether resizing is required. The model pipeline resizes all images to a consistent input size.

6. Preprocessing and Dataset Split

Each image is read from its filepath, decoded as a three-channel RGB image, resized to 128 × 128 pixels, converted to floating-point values, and divided by 255.0. This normalizes pixel values to the 0–1 range.

Class labels are mapped to integer IDs and then converted into one-hot vectors for nine-class classification. Stratified sampling approximately maintains class proportions in each subset.

Expected approximate split for 2,357 images: training 1,649 images (about 70%); validation 236 images (about 10%); testing 472 images (about 20%). Confirm exact counts from the notebook output.

7. CNN Architecture

The model uses a Keras Sequential architecture:

Input: 128 × 128 × 3 RGB image.

Conv2D: 32 filters, 3 × 3 kernel, ReLU activation, same padding; followed by 2 × 2 max pooling.

Conv2D: 64 filters, 3 × 3 kernel, ReLU activation, same padding; followed by 2 × 2 max pooling.

Conv2D: 128 filters, 3 × 3 kernel, ReLU activation, same padding; followed by 2 × 2 max pooling.

Flatten layer to convert feature maps into a vector.

Dense layer with 128 neurons and ReLU activation.

Dropout layer with a rate of 0.5 to help reduce overfitting.

Output dense layer with nine neurons and softmax activation.

Softmax produces a probability distribution across the nine classes. The class with the highest probability becomes the predicted label.

8. Compilation and Training

The model is compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy as a metric. Categorical cross-entropy is suitable for a multi-class task with one-hot labels.

The model is trained with batches of 32 images for up to 15 epochs. Validation data is checked after each epoch. EarlyStopping monitors validation loss, stops after three epochs without improvement, and restores the weights from the best validation-loss epoch.

The notebook displayed a CUDA initialization warning. The image pipeline nevertheless ran and reported 52 training batches, 8 validation batches, and 15 testing batches. CPU execution is possible when a working GPU is unavailable, although it may be slower.

9. Training Results

The training output shared during the project showed the following values at the displayed epoch:

Training accuracy: 60.28%. Training loss: 1.0840. Validation accuracy: 55.51%. Validation loss: 1.2750.

These are the metrics for the displayed epoch, not the final test accuracy. Record the final test accuracy and test loss from the output of model.evaluate(test_ds).

10. Evaluation Metrics

Accuracy is the proportion of predictions that are correct. Loss measures the error used during optimization. Precision measures how many predictions assigned to a class are correct. Recall measures how many actual examples of a class are identified. The F1-score is the harmonic mean of precision and recall.

A confusion matrix compares actual labels with predicted labels. Values on the main diagonal are correct predictions; off-diagonal values show which classes are confused. Since class counts are uneven, macro-average and per-class metrics are useful alongside overall accuracy.

Complete these values from the actual notebook output: Test accuracy: [insert value]; test loss: [insert value]; best-performing class: [insert from report]; class needing most improvement: [insert from report].

11. Sample Predictions

The notebook displays up to ten held-out test images with their actual labels and predicted labels. Review both correct and incorrect examples. Model confidence should not be interpreted as medical certainty.

Insert or retain the screenshot of the ten-image prediction grid from the Kaggle notebook in the final submission.

12. Limitations

The dataset is relatively small for a nine-class image task.

Class imbalance may bias predictions toward more frequent categories.

Resizing images to 128 × 128 pixels may remove fine visual details.

The model may learn dataset-specific patterns and may not generalize to images from other cameras, clinics, or populations.

Results can vary with the random split and training settings.

The model has not been clinically validated and must not be used to diagnose or rule out skin cancer.

13. Future Improvements

Potential improvements include data augmentation, class weights, transfer learning with a pretrained image model, and careful hyperparameter tuning. Performance can also be assessed using macro-F1, balanced accuracy, and per-class recall. Improvements should be compared on the same held-out test set without repeatedly tuning against it.

14. Conclusion

This project demonstrates an introductory deep-learning workflow for classifying skin-lesion images into nine dataset categories. It includes EDA, image preprocessing, a stratified 70/10/20 split, CNN construction, training, accuracy/loss plots, test evaluation, a classification report, a confusion matrix, and sample predictions. The displayed training run reached 60.28% training accuracy and 55.51% validation accuracy at the reported epoch. The final evaluation results should be copied from the notebook before submission. The project provides practical experience with computer vision while highlighting the limitations of medical-image models.

References

TensorFlow/Keras documentation: https://www.tensorflow.org/api_docs/python/tf/keras

scikit-learn documentation: https://scikit-learn.org/stable/

Kaggle dataset used in the notebook: Skin Cancer ISIC The International Skin Imaging Collaboration. Add the exact dataset page URL required by your instructor.

Course lectures and laboratory material on EDA, CNNs, and image classification.

Appendix A: Final Submission Checklist

• Add the EDA class-distribution chart and sample-image screenshot.

• Add training/validation accuracy and loss graphs.

• Insert final test accuracy and test loss from the notebook.

• Add the classification report and confusion matrix.

• Add the ten-image prediction grid.

• Fill in student, course, instructor, university, and submission details.